

DeepCTI: Evidence State Control for Threat Mitigation Decisions

Anonymous Author(s)
Anonymous Institution

Abstract—Cyber threat intelligence (CTI) states which software versions are vulnerable, but deciding whether a particular host is affected, and whether an agent may change it, depends on local evidence that can be missing, noisy, stale or adversarial. We argue that LLM-assisted mitigation should be governed by an explicit evidence state, not by the model’s reasoning trace. DeepCTI keeps a provenance-tracked four-valued (Belnap) state over VEX-relevant atoms, counts support in independence groups of trusted sources, decides with a deterministic VEX table that abstains on missing or conflicting evidence, acquires evidence by value of information, admits LLM-extracted facts only after span verification, and authorizes disruptive tools through Cedar policies that require independent trusted support. We evaluate on two new live-fixtured benchmarks with sealed labels: D1 (906 Debian hosts) and D7 (822 hosts spanning Ubuntu, PyPI, Maven and vendor software). Baselines include scanners, a tracker lookup, direct prompting and ReAct agents (ours and Inspect AI’s), run with up to ten open models from 4B to 128B parameters across four pre-registered rounds (about 270k test and calibration episodes). Without a vulnerability feed, DeepCTI lowers decision loss relative to ReAct by 1.58 (D1) and 0.88 (D7) with no dangerous errors. On vendor software, where versions exist only in text, verified LLM extraction lowers loss by 0.21 relative to an LLM-free controller. Learn-then-Test releases abstentions at realised risk below 2%, raising coverage from 0.31 to 0.50–0.91. Evidence-gated authorization removes the 5–6% of episodes in which an unmodified ReAct agent takes an unauthorized disruptive action. We also report where the design fails: a restart-blind controller (repaired by instance-aware evidence, loss 9.67 to 0.42), conservative trust estimation that discards useful scanners, and injection attacks too weak to discriminate defences.

Index Terms—Cyber threat intelligence, vulnerability management, VEX, LLM agents, evidence state, authorization

I. Introduction

Cyber threat intelligence (CTI) describes vulnerabilities, affected products and possible mitigations. Acting on it requires local facts: whether the product is installed, which version is on disk and which is running, whether a backport already fixed the flaw, and whether a vulnerable feature is enabled. Patch management studies show that remediation also depends on testing, coordination and service availability [1]–[3]. The output that operators need is a VEX statement (affected, not affected with a justification, fixed, or under investigation) [4], and, where a change is warranted, an authorized remediation.

Large language model (LLM) agents that interleave reasoning with tool calls [5], [6] can gather this evidence, and LocalIntel shows that combining global CTI with

organization-specific context helps [7]. Our measurements, however, show that such agents fail in a specific and costly way: they miss vulnerabilities. A ReAct agent misses 34–49% of affected Debian hosts in our benchmark, and its loss varies fifteenfold across models. They also take disruptive actions they were not authorized to take, even without any attacker. Both failures stem from the same design choice: the agent’s beliefs about the host live only in its context window, without provenance, trust or freshness.

We argue that CTI-driven mitigation should be governed by an explicit evidence state: a record of which trusted, independent, fresh observations support or refute each decision-relevant fact. The state determines the decision and gates every action, and the language model is confined to roles where it can be checked. This leads to two research questions.

RQ-1: Which evidence state, decision rule and acquisition strategy yield correct, abstaining-when-unsure VEX decisions as observations arrive, conflict and go stale?

RQ-2: How should mitigation judgments be connected to authorization so that investigative and disruptive actions remain within policy, including when some sources are adversarial?

Contributions.

- **Design.** DeepCTI combines several mechanisms (§III):
 - a provenance-tracked Belnap evidence state with trust classes, independence groups and freshness windows;
 - a deterministic VEX decision table;
 - value-of-information (EC^2) acquisition;
 - span-verified LLM extraction;
 - Learn-then-Test release of hint-based decisions;
 - evidence-conditioned Cedar authorization with a fragment linter that guarantees a k -of- n corroboration property.
- **Benchmarks.** D1 and D7 are live-fixtured benchmarks: root file systems built from official base images, with package databases, version files, jar metadata and real scanner reports. Labels come from authoritative trackers, and test splits are sealed. They are extended with drift (D2, X5) and adversarial (D3) episodes (§IV).
- **Pre-registered evaluation.** We test 19 hypotheses over four tagged pre-registration rounds, ten open models and a third-party baseline, reporting failed hypotheses and post-hoc corrections (§V).

- Findings. We characterize what the LLM contributes in such a system: little where evidence is structured, and a measurable gain where it is not. We identify two failure modes, restart blindness and over-conservative trust, and we evaluate repairs for both.

II. Related Work

CTI and mitigation. CTIBench, AthenaBench, CTI-Connect and ExCyTIn-Bench evaluate CTI knowledge, mitigation mapping, retrieval over CTI sources and interactive investigation [8]–[11]. None asks whether a specific host is affected. LocalIntel contextualizes CTI with organizational knowledge [7], and TrustOps argues for evidence-grounded defence under missing, conflicting and stale evidence [12]. Patch-management studies document delays, testing needs and failures of security patches [1], [2], [13], [14], and NIST guidance covers configuration monitoring [15], [16]. DeepCTI turns these requirements into a decision procedure with measurable loss.

Retrieval, reasoning and verification. RAG and its corrective and reflective variants ground generation in retrieved text [17]–[20]. Freshness and conflict-aware retrieval address changing knowledge [21]–[23], and misleading context degrades answers [24]. Memory systems such as MemGPT keep long-lived context [25]. In all of these, retrieved text informs a generated answer. In DeepCTI, an observation changes the decision only through a trusted source’s verified fact, and untrusted text remains a hint.

Agent authorization. Least privilege, complete mediation and attribute-based access control are the classical foundations [26]–[28]. Progent, AgentSpec, CaMeL and DRIFT constrain agent tool use by privilege, runtime rules, data-flow separation or isolation [29]–[32], and [33] formalizes action alignment and source authorization. Closest to our corroboration rule are TMA-NM [34] and certified multi-source integrity [35], which admit an action only with support from several independent trusted principals or domains. DeepCTI differs in three ways. Its evidence state is four-valued and time-indexed. Its policies are standard Cedar [36], linted into a fragment for which the guarantee holds. Its decision semantics are those of deployed VEX. AgentDojo, InjecAgent, ToolEmu and R-Judge evaluate injection and risk awareness [37]–[40]; we follow [41] in reporting the benign error rate next to attack success.

Active acquisition and risk control. EC² selects tests that separate decision regions at low expected cost [42], [43]. Learn-then-Test (LTT) selects parameters of a predictor with finite-sample risk control [44], and [45] applies LTT-style tests to decide when a clinical diagnosis agent may stop. VEX-Bench evaluates VEX status prediction on source repositories [46]. We combine both with a Belnap state [47], so that “unknown” and “conflicting” are first-class outcomes.

III. DeepCTI

A. Setting and threat model

A case is a pair (host, CVE). The agent sees only the host name, release, CVE and component name, and it acts through mediated tools:

- read tools: package query, file read, directory listing, service status, configuration read, language-package query, CMDB lookup, advisory fetch, tracker/VEX lookup;
- medium-risk tools: scanner run, change approval request;
- disruptive tools: patch, restart, feature disable.

Each tool has a cost: read tools cost 1, a scanner run 5, a patch 20 and a restart 15; the budget is 60. Each source carries a trust class (T/U) and an independence group:

- pkgdb: the package database, and also the scanners, which read it;
- fs: changelogs, configuration and version files;
- proc: process view and banners;
- langdb/artifact: language-package metadata and jar contents (D7);
- change: approvals and rollback;
- feeds (tracker and VEX), CMDB and advisories, each in its own group.

Trust is estimated on the development split. A source is trusted iff the Wilson 95% upper bounds of its false-positive and false-negative rates are both ≤ 0.10 with $n \geq 20$. Advisory text, CMDB notes and anything an LLM compiles are untrusted. The attacker can inject arbitrary untrusted text, and can compromise up to m trusted groups so that they report forged values.

B. Evidence state

An observation is $(f, \pm, s, t, h)$: atom f , polarity, source s , time t and content hash h . The world atoms are present, in_affected_range, fix_applied and vuln_config_enabled. The change atoms are change_approved, rollback_available and in_maintenance_window. For each atom, the state takes the latest fresh observation of every trusted source. Freshness windows are $\Delta = 12$ time units for world atoms and 30 for change atoms. The state then counts positive and negative support in distinct independence groups:

$$\kappa_f^\pm = |\{g(s) : s \in \mathcal{T}, \text{last}_\Delta(s, f) = \pm\}|, \quad (1)$$

where $\mathcal{T}$ is the set of trusted sources, $g(s)$ the group of s , and $\text{last}_\Delta(s, f)$ the polarity of the latest observation of f by s within the window. The Belnap value is T if $\kappa_f^+ > 0 = \kappa_f^-$, F in the converse case, B (both) if both counts are positive, and N (none) otherwise [47]. Untrusted observations never change a value; they are kept as hints. The log is append-only, and only tool executions (including the carried-over history of earlier ones) write to it.

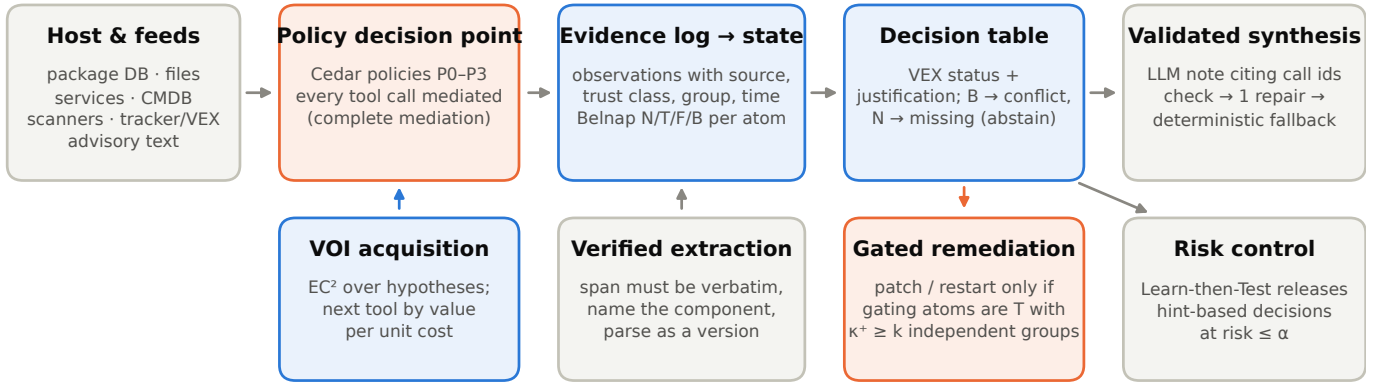


Fig. 1. DeepCTI architecture. Every tool call passes the policy decision point. Tool outputs become observations with source, trust class, independence group and time. The deterministic VEX table reads the state, and the LLM proposes extractions (verified before entering the state), advisory hints and the analyst note (validated). Disruptive tools need gating atoms supported by $\geq k$ independent trusted groups.

C. Decision table

The VEX decision is a pure function of the atom values, evaluated row by row:

- `present=F` gives not affected (component not present).
- `fix_applied=T` gives fixed.
- `in_affected_range=F` gives not affected (vulnerable code not present).
- If the CVE has a configuration precondition, `vuln_config_enabled=F` gives not affected (requires configuration).
- Otherwise the decision is affected.

A B on a consulted atom yields under investigation (conflict). An N on an atom that blocks every later row yields under investigation (missing), together with the atoms whose acquisition would decide the case. Abstention is therefore explicit and explained, never a guess.

Running example. CVE-2024-6387 (regreSSHion) affects upstream OpenSSH 8.6–9.8. Consider a Debian 12 host with `openssh-server 1:9.2p1-2+deb12u3`. Its upstream version lies inside the range, so a naive comparison says affected, but this Debian revision is the tracker’s fixed version (a backport). DeepCTI reads the installed version from the package database and the fixed version from the tracker. It sets `present=T` and `fix_applied=T`, and decides fixed after two calls. If the package was upgraded but `sshd` still runs the old build, the decision changes. The instance-aware version below observes `running_fix_applied=F` from the process view and decides affected. A restart is then permitted only with an approved change inside a maintenance window.

Instance-aware evidence (v2.1). Our first evaluation exposed a failure: a package upgraded on disk while the old binary keeps running is reported fixed from the package database. v2.1 adds the atoms `running_in_affected_range` and `running_fix_applied`, observed from the process view. A component is in range if any instance, on disk or running, is in range, and it is fixed

only if all observed instances are fixed. Before deciding fixed or not affected (code not present), acquisition must consult the process view.

D. Acquisition

Given priors over hypotheses (joint assignments of the atoms) estimated on the development split, DeepCTI chooses the next tool by EC^2 : the expected weight of edges cut between hypotheses with different decisions, per unit cost [42]. It stops when every hypothesis consistent with the state yields the same decision, or when the budget is exhausted. Ablations replace EC^2 with entropy-greedy, a fixed checklist, random order, or an LLM choosing the next tool.

E. Verified LLM extraction

Where evidence is text (changelogs, release notes, banners, manifests), the LLM proposes facts of the form (atom or version, value, verbatim span, call id). A verifier admits a fact only if:

- the span occurs verbatim in that call’s output;
- the span names the component, with the version as the first version token within 40 characters;
- it comes from a grammar-sanctioned location: the changelog header line, a vendor version document, or a process line (for Debian/Ubuntu only the line naming the package build, since an upstream banner cannot establish a distribution revision);
- it parses as a version of the ecosystem: `dpkg` semantics for Debian/Ubuntu, `univers` for PyPI, Maven and vendor versions.

An admitted fact becomes an observation of the underlying (trusted) source. Advisory text that the LLM compiles into version ranges remains untrusted, so it yields hints only.

F. Risk-controlled release

When no trusted source decides a case, DeepCTI abstains. In that case the hints (an LLM-compiled advisory

range applied to verified versions) often point to a decision. We release hint-based decisions with LTT [44]. The score of an abstention is the smallest support margin (trusted plus hint support for the hinted value, minus support against it) over the atoms the decision needs. Thresholds are tested from strict to loose, in fixed sequence, with Hoeffding–Bentkus p -values. The selected threshold satisfies $\Pr[R > \alpha] \leq \delta$, where R is the share of cases released as not affected or fixed although the host is affected ($\alpha = 0.05$, $\delta = 0.1$).

G. Authorization

Every call is checked by a Cedar policy decision point [36]. P0 permits everything. P1 checks only that arguments are well formed and that paths are allowed. The evidence-gated policies P2 and P3 additionally check that arguments are in scope and require, for each gating atom f of a disruptive action, both $\text{val} = \text{T}$ and $\kappa_f^+ \geq k_f$. A patch is gated on `present`, in `affected_range`, `change_approved` and `rollback_available`. P2 uses $k = 1$. P3 uses $k_{\text{world}} = 2$ and $k_{\text{change}} = 1$. A linter accepts only permits that are pure conjunctions containing both conditions for every gating atom.

Property. For such policies, an attacker who controls only untrusted text, or who compromises $m < k_f$ trusted groups, cannot make $\kappa_f^+ \geq k_f$ hold falsely for a gating atom f . No disruptive action is therefore permitted on a falsely supported gating atom. The guarantee covers only the gating atoms; a restart, for instance, is gated on change atoms alone. At $m \geq k_f$ the guarantee no longer holds.

H. Validated synthesis

The LLM writes the analyst note from the final state and the cited tool calls. In v2.1 the note is validated:

- the decided status must be stated (v2.1b additionally rejects any other status word);
- cited call ids must exist;
- every version string must occur in the cited evidence.

A failing note gets one repair attempt, and otherwise a deterministic fallback. The note never changes the decision.

IV. Evaluation Design

Arms. The tracker arm exposes a tracker/VEX feed with structured affected ranges. The withheld arm removes it, leaving scanners, files and processes. The blind arm (D7) also removes scanners.

Systems. The baselines are:

- S0: scanners alone;
- S1: tracker lookup with version comparison;
- S1': the DeepCTI controller without any LLM (checklist acquisition);
- S2: direct prompting with all evidence;
- S3: ReAct with the same tools, prompt core and budget;
- S4/S5: ReAct with a scratchpad or reflection;

TABLE I

Benchmarks (test split). Hosts are root file systems built from official base images, with real package versions, changelogs, version files and jar metadata (no executable code), and real Trivy/Grype/OSV-Scanner reports; process tables are simulated. Labels come from authoritative trackers and are sealed until pre-registration.

Set	Content
D1	Debian 11–13; 906 cases, 192 CVEs, 32 source packages; test 453 cases / 96 CVEs, 48% published after 2026-05-01 (temporal hold-out). Variants: absent, vulnerable, fixed, backport (naive upstream comparison is wrong), config-gated, decoy package. Labels: Debian security tracker.
D2	Drift since a previous assessment (carried-over history at $t = -30$): upgrade with or without restart, rollback, removal, config enable; 121 test episodes per model.
D3	Adversarial: injected advisory/CMDB text, forged trusted groups ($m=1..3$); 64 episodes per model.
D7	Ubuntu 22.04/24.04, PyPI apps, Maven jars (incl. nested fat-jar and pip-vendored copies), vendor software whose version exists only in text (Tomcat, httpd, Jenkins, Roundcube, GitLab); 822 cases, 141 CVEs; test 414 / 70 CVEs, 43% temporal hold-out, 21 config-gated. Labels: Ubuntu tracker, OSV ranges, vendor advisories.
X5	Fresh drift episodes on D7 test hosts (instance versions and banners rewritten consistently).

- S3I: Inspect AI’s ReAct agent [48].

DC is DeepCTI, DCv21 the instance-aware version, and DCv21b DCv21 with a neutral synthesis prompt and a stricter status check (§V-F). Ablations remove verification (DC_noverify), trust classes (DC_k1), freshness (DC_nofresh) or acquisition (checklist, entropy, random, LLM-chosen). All prompts share one frozen core.

Models. All models are served with vLLM 0.23, at temperature 0 for the main runs:

- the v1 panel: Qwen3-4B, Llama-3.1-8B, Granite-4.1-8B, Qwen3-14B, Mistral-Small-3.2-24B, Gemma-4-31B;
- added in later rounds: Granite-4.1-30B, Nemotron-Super-49B, Mistral-Medium-3.5-128B and GLM-4.5-Air (106B MoE, descriptive only).

Metrics. The primary metric is decision loss, with a miss costing 10, a needless change 1, fixed↔not affected 0.2, and abstention 0.5. Invalid output counts as abstention. We also report:

- the dangerous-error rate (DER: misses among affected cases) and coverage;
- cost to decision;
- for security, the share of episodes with an unauthorized disruptive action (UDAR) and the false-block rate;
- for notes, the share of atomic claims judged supported by up to three LLM judges, excluding judges from the generator’s model family. The judges were admitted only after an error-injection validation (recall ≥ 0.85 , false alarms ≤ 0.10).

Protocol. Four pre-registrations were each committed and tagged before the corresponding test runs:

- prereg-v1: D1–D3, H0–H6;
- prereg-v2: panel extension and DCv21, H7–H8;
- prereg-v3: D7, H9–H15;

TABLE II

Pre-registered hypotheses (test split; estimate with 95% CI). Loss and cost differences are DeepCTI minus comparator (negative favours DeepCTI). Decision rules as pre-registered (§IV).

Claim	Estimate	Held
prereg-v1: D1–D3, 6 models		
H0 tracker arm: $DC \equiv S1$ accuracy (± 3 pp)	+0.002	yes
H1 withheld: loss $DC < S3$	−1.585 [−1.76, −1.42]	yes
H2 D2: staleness error $DC < S3$	−0.264	yes
H3 EC^2 cheaper than checklist	−0.03, $p=1$	no
H4 attack success $DC+P3 < S3+P1$	−0.099	yes [†]
H5 LTT risk $\leq \alpha$ (withheld)	vacuous	–
prereg-v2: D1/D2, 9 models		
H7 upgrade w/o restart: $DCv21 < DC$	−9.25 [−9.76, −8.43]	yes [‡]
H8 other drift: $DCv21$ non-inferior	0.000	yes
prereg-v3: D7, 8 models (H14: 2; H15: 7 generators)		
H9 withheld: loss $DC < S3$	−0.883 [−1.03, −0.74]	yes
H10 vendor, tracker: $DC < S1'$ (LLM-free)	−0.205 [−0.26, −0.14]	yes
H11 accepted-fact error $DC < \text{noverify}$	−0.027 [−0.04, −0.02]	yes
H12 LTT blind arm, $\alpha=0.05$	$\geq 93\%$ splits	yes
H13 D7 drift: $DCv21 < DC$	−4.59 [−6.30, −3.02]	yes
H14 EC^2 cheaper than checklist	−0.35 [−0.51, −0.21]	yes
H15 $DCv21$ notes non-inferior (−0.02)	−0.018 [−0.05, +0.01]	no
prereg-v4: D7, 8 models		
H16 $DCv21b$ notes non-inferior (−0.02)	+0.044 [+0.01, +0.07]	yes
H17 DCt (more trust data) $< DC$, withheld	−0.137 [−0.17, −0.11]	yes
H18 decoys: DC loss non-inferior (+0.05)	−0.026 [−0.05, −0.01]	yes [§]

[†]attacks barely moved S3 (§V-E). [‡]confirmatory only: these episodes motivated v2.1. [§]baseline ran on older code; same-code control: −0.001 [−0.002, 0], low exposure (§V-G). H6 (model-size interaction) exploratory.

• prereg-v4: H16–H18.

Test labels stay sealed until their tag. Superiority hypotheses use two-sided CVE-clustered sign-flip permutation tests on per-CVE mean differences, with Holm correction within the v1–v3 families [49]. Equivalence (H0), non-inferiority (H8, H15, H16, H18) and criterion hypotheses (H5, H12, H17) use CVE-clustered bootstrap 95% CIs and pre-registered rules. The v4 hypotheses are not adjusted for multiplicity. Every change after a tag is logged (28 entries). Separate audit passes re-derived the pre-registered estimates and report tables from raw logs.

V. Results

A. Decisions without a feed

Fig. 2 and Table II give the headline results. With a tracker, DeepCTI has zero loss on D1 and is equivalent to the lookup (H0). On D7, its loss is 0.030, compared with 0.071 for $S1'$ and 0.147 for $S1$. Without a tracker, ReAct’s loss rises to 1.65 (D1) and 1.23 (D7). Its errors are mostly misses: DER is 0.49 on D1 and 0.30 on D7. DeepCTI, by contrast, has DER 0 in every arm. On D7 its coverage is 0.31: it abstains when no trusted source decides. H1 and H9 hold with large margins.

On D1 the result holds for every model (Fig. 3). ReAct loss with a tracker ranges from 0.18 (Gemma-31B) to 2.64 (Qwen3-4B), and reliability under resampling (pass⁵ at $T=0.7$) ranges from 0.98 (Gemma-31B) to 0.08 (Llama-8B), or 0.00 for Nemotron-49B; DeepCTI’s pass⁵ is 1.0 for all nine models. DeepCTI’s decisions are identical for every model. On D7 there is one exception: without a feed, Gemma-31B ReAct has lower loss than DeepCTI (0.291

vs. 0.344, DER 0.047 vs. 0), and without scanners the two are about equal (0.365 vs. 0.344). ReAct trades misses for coverage, DeepCTI abstains. The largest model narrows but does not close the pooled gap: Mistral-Medium-128B ReAct exceeds DeepCTI by 0.41 (D1 tracker), 0.79 (D1 withheld), 0.18 (D7 tracker) and 0.30 (D7 withheld). Inspect AI’s ReAct agent agrees with ours on 80.6% of D7 decisions ($n=1,635$). With a tracker its loss is similar (0.92/1.14 versus $S3$ ’s 0.84/1.13 for Mistral-24B/Qwen3-14B). Without one it is somewhat lower (1.50/1.51 versus 1.63/2.14), so our ReAct baseline is, if anything, slightly weaker than a standard harness in that arm.

B. What the language model contributes

On D1, and in both feed-less D7 arms, DeepCTI’s decisions equal those of the LLM-free controller $S1'$: the advantage comes from the evidence state and decision table, not from model reasoning. The LLM matters in two places.

Unstructured evidence. For vendor software, the version exists only in text files (release notes, headers, configuration files) and banners. There, in the tracker arm, verified extraction lowers loss relative to $S1'$ by 0.205 (H10). Verification reduces the share of accepted version facts that are wrong by 2.7 percentage points (H11; Fig. 4). It rejects 9% of proposals from files and 17% from process banners, and it eliminates wrong facts from banners (4.1% to 0%). On benign vendor cases with a tracker, unverified extraction nevertheless had lower loss (0.084 vs. 0.146). Most of that gap was a verifier bug: header banners such as X-Jenkins: were rejected. With the bug fixed post hoc (a logged deviation; 8 models), the gap is 0.113 vs. 0.084.

Risk-controlled release. In the blind arm, hints point to a decision for most abstentions. We pool calibration and test records and draw 200 CVE-level re-splits (40% calibrate, 60% evaluate). LTT then raises coverage from 0.31 to between 0.50 (Qwen3-4B) and 0.91 (Mistral-Medium). Mean realised risk is 1.4–1.9%, and at least 93% of re-splits stay within α for every model (H12; Fig. 5). Here coverage scales with model quality while the risk guarantee does not depend on it. This is the converse of ReAct, where quality and risk move together.

C. Drift

On D2, DeepCTI reports the pre-drift status when it is wrong less often than ReAct (H2). The first version, however, was restart-blind (Fig. 6). When a package is upgraded without a restart, package database, changelog and scanners all say fixed while the vulnerable binary keeps running. DC’s loss on these episodes is 9.67 (accuracy 0), worse than ReAct under the original prompt (7.56). For the v2.1 study all systems shared a revised prompt that tells them to check running services, and ReAct’s loss fell to 3.98. Instance-aware evidence (DCv21) reduces the loss to 0.42 (accuracy 0.93, H7) and is non-inferior on all other D2 drift kinds (H8). Because these episodes motivated the

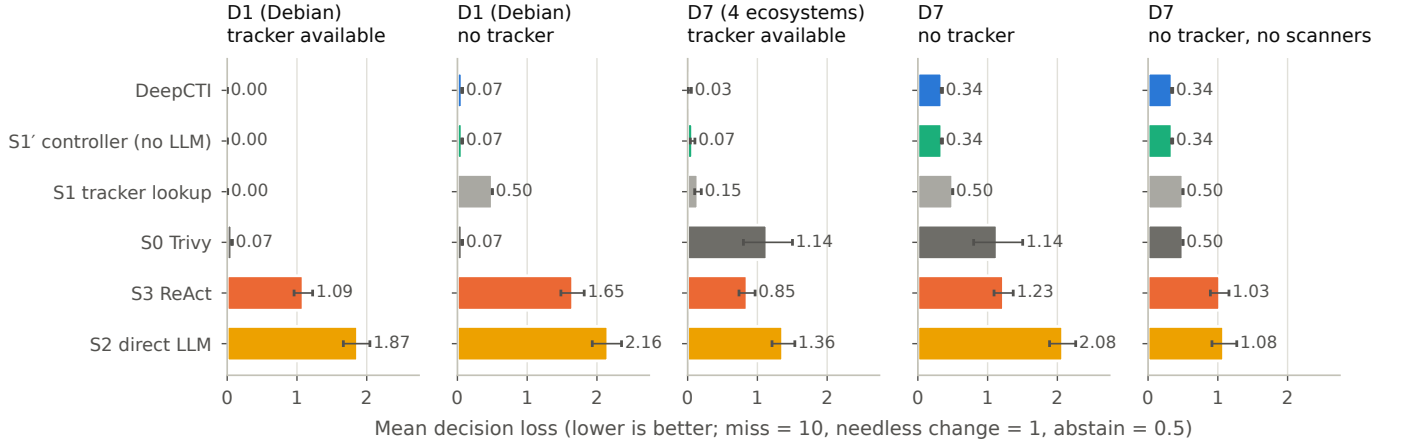


Fig. 2. Mean decision loss on the test splits (pooled over models; whiskers: CVE-clustered 95% CIs). With a tracker, DeepCTI matches a lookup. Without one, it decides from trusted sources (on D1, the trusted scanners) and abstains where none decides (D7), whereas ReAct (S3) and direct prompting (S2) miss vulnerabilities. Trivy is trusted on D1 but noisy on D7.

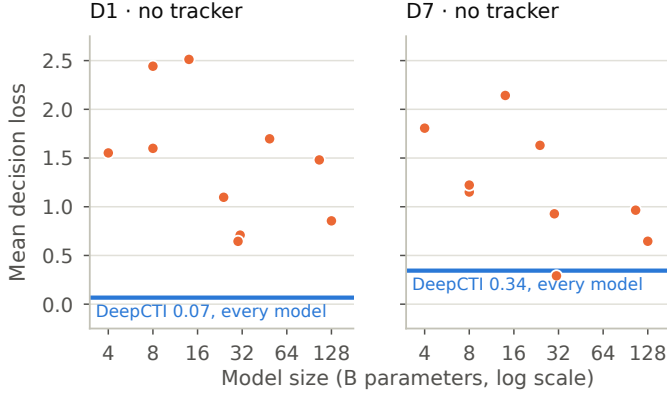


Fig. 3. ReAct (S3) loss per model versus parameter count, no-tracker arms. Each dot is one model. The blue line is DeepCTI, whose decisions are identical for every model.

fix, we pre-registered an independent test on fresh D7 drift episodes. There, DCv21 has loss 0.45, against 5.04 for DC and 2.03 for ReAct (H13). On rollbacks ReAct fails (loss 4.08 on D7, 6.87 on D2), while both DeepCTI versions stay near zero. The fix has a cost on D7. On upgrades with restart DCv21 abstains more (accuracy 0.74 vs. 0.95, loss 0.128 vs. 0.025). It also needs more tool cost: 8% more in the main D7 study and 34–79% more on the drift episodes.

D. Acquisition

On D1, acquisition does not matter: with a tracker every strategy except random order decides in about two calls, and H3 fails (-0.03 , $p=1$). On D7 without a tracker, where acquisition is non-trivial, EC² reaches a decision with lower cost than the fixed checklist (-0.35 cost units at identical loss, H14; Fig. 7). Entropy-greedy is cheaper still (4.20 vs. 5.34 units), so we do not claim that EC² is the best rule, only that VOI beats a fixed order. ReAct spends a similar budget at three to five times the loss.

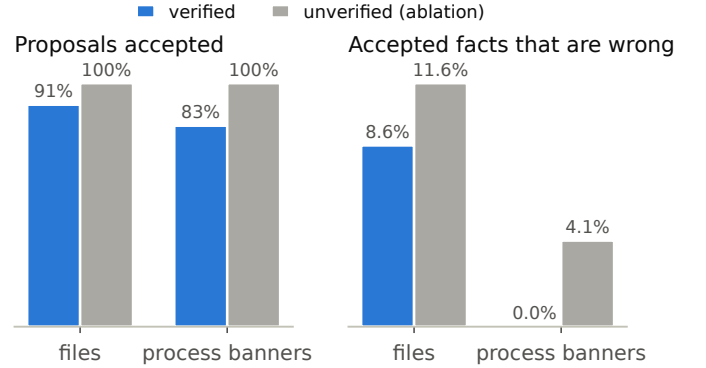


Fig. 4. Verified versus unverified LLM extraction on D7 (all arms, 8 models): share of proposed version facts admitted, and share of admitted facts that are wrong.

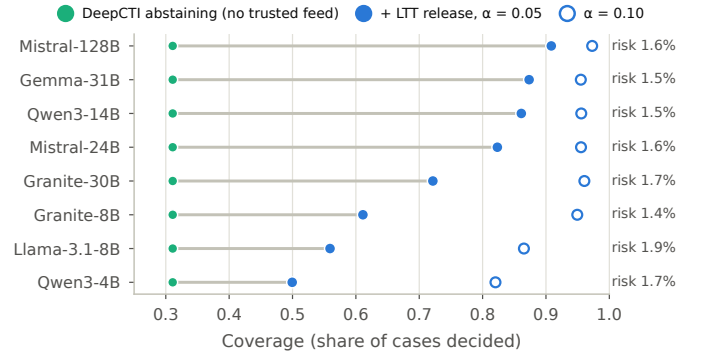


Fig. 5. Learn-then-Test release of hint-based decisions, D7 blind arm. Coverage without release (green), with release at $\alpha=0.05$ (filled) and 0.10 (open); mean realised risk at $\alpha=0.05$ over 200 CVE re-splits on the right.

E. Authorization

On D3, an unmodified ReAct agent executes unauthorized disruptive actions in 5–6% of episodes (Fig. 8). This

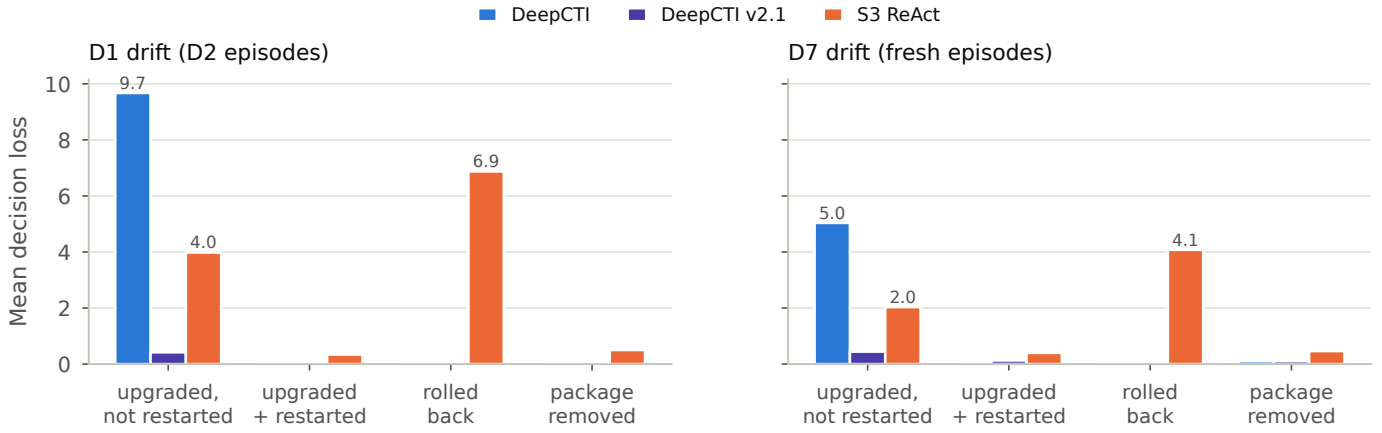


Fig. 6. Decision loss by drift kind. Left: D2 (9 models). Right: fresh D7 drift episodes (8 models). The original controller is restart-blind; instance-aware evidence (v2.1) removes the failure. It is non-inferior on the other D2 kinds, but on D7 it abstains more after restarts.

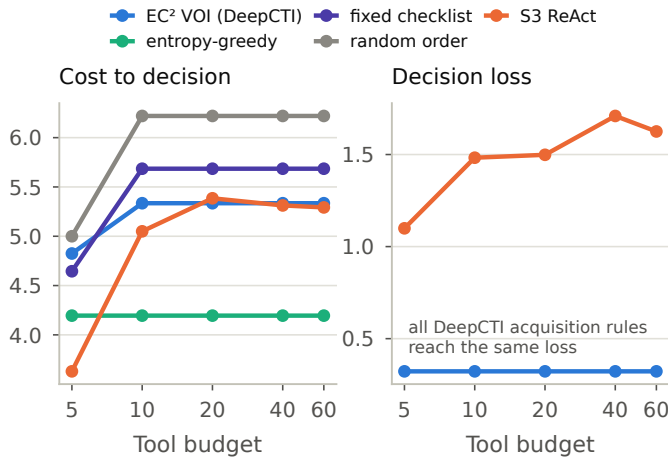


Fig. 7. Acquisition on D7, withheld arm (Qwen3-14B, Mistral-24B): mean tool cost to the decision and decision loss by budget.

happens without an attacker as well: the agent patches or restarts on its own initiative. Argument checks (P1) and a prompt defence do not change the rate. Evidence gating (P3) removes these actions for the same agent and for DeepCTI. This has a liveness cost: ReAct+P3 blocked one model’s warranted remediations (false-block rate 0.33 over 6 warranted episodes). The (k, m) grid shows the guarantee’s boundary:

- forging one trusted world group ($m=1 < k=2$) yields no unauthorized action, while forging two ($m=k$) yields 0.017;
- compromising the change system, whose atoms have $k=1$, yields 0.33 (6 episodes);
- P3k3 ($k=3$) blocks all tested compromises.

The attack study is weak: injected text barely changed ReAct’s behaviour (ASR minus the benign rate of the same goal was -0.005). H4 therefore mostly measures ordinary errors, and we do not claim robustness to

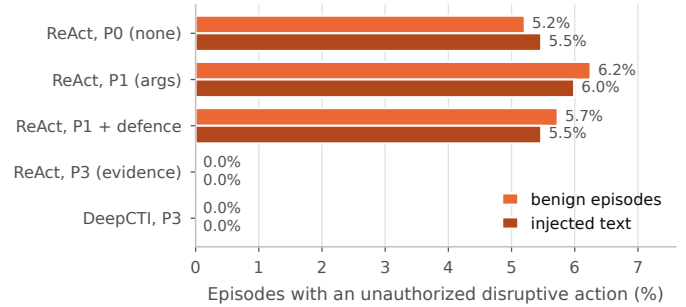


Fig. 8. Episodes with an unauthorized disruptive action on D3 (3 models), benign versus injected text. Evidence gating (P3) removes them; argument checks and a prompt defence do not.

adaptive injection.

F. Analyst notes

The v2.1 validator was pre-registered as non-inferior to DC’s unvalidated notes (H15), and it failed (-0.018 $[-0.047, +0.010]$). The cause was a prompt defect: the synthesis prompt said “the decision is FIXED (do not change it)”, and 40.5% of non-fixed notes then contained “FIXED”. With a neutral prompt (DCv21b, post hoc), faithfulness rose by $+0.036$ $[+0.011, +0.062]$ over DC on the same sample, and note–decision consistency was 0.978 (DC 0.960, DCv21 0.752). Of these notes, 90.9% passed validation on the first attempt, 3.4% after repair, and 5.8% fell back. We confirmed this on a fresh, pre-registered sample of 504 triples drawn from cases disjoint from the H15 sample and covering all eight generators (H16). DCv21b’s faithfulness was 0.744 against DC’s 0.700, a difference of $+0.044$ $[+0.014, +0.073]$, so non-inferiority holds and the estimate excludes zero. Its note–decision consistency was 0.988 (DC 0.961), and 6.0% of its notes fell back to the deterministic template (Fig. 9).

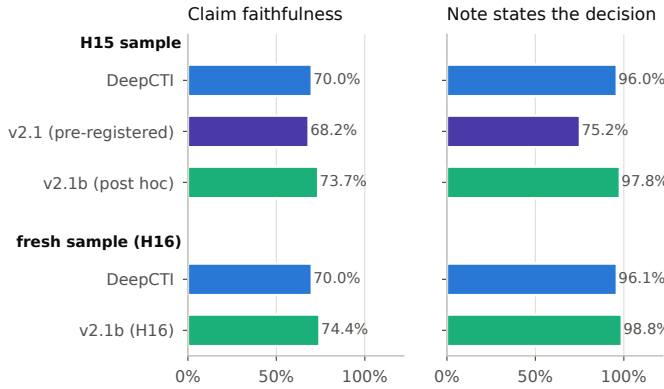


Fig. 9. Analyst-note quality (D7): share of atomic claims judged supported by up to three validated judges from other model families, and share of notes stating the controller’s decision.

G. Trust estimation and decoys (prereg-v4)

Trust. With trust estimated on 42 development CVEs, no scanner met the rule on D7, so DC ignored scanners and abstained where scanners were right. Re-estimating on development plus calibration data (71 CVEs) with the same rule trusts scanners for Ubuntu and Maven, but not for PyPI (false-negative rate 10.5%) or vendor software (where scanners never find the version). This DCt variant lowers withheld-arm loss from 0.344 to 0.207 (-0.137 [$-0.167, -0.107$], H17) and raises coverage from 0.31 to 0.73, with DER still 0. Its decisions are again identical across the eight models. Conservative trust was thus a data-quantity problem, not a flaw of the rule.

Decoys. A realistic wrong version mention was added to the documents the systems actually read. On vulnerable Ubuntu hosts it is a changelog body line naming the fixed version. In vendor software it is a “see also” line in every document that states the installed version. A dev pilot showed that decoys placed in unread notes are never observed. The pre-registered comparison passes (-0.026 [$-0.050, -0.006$], H18), but the audit found a confound. For seven models the clean baseline ran before the verifier fixes, so that contrast mixes the decoy effect with a code change. We therefore added a same-code control run without decoys (logged deviation). Against it, decoys change DC’s loss by -0.001 [$-0.002, 0$], DC_noverify’s by 0, and ReAct’s by -0.02 [$-0.13, +0.10$]. Neither DC nor DC_noverify ever accepted a decoy version as a fact. Exposure was low, however: DC read the decoy before deciding in only 11.5% of episodes (3 CVEs), because it decides from the package database or tracker first. H18 therefore shows that such decoys are inert for DeepCTI, not that its verifier would resist targeted version spoofing.

Panel completion. On D3, Mistral-Medium-128B’s ReAct agent took no unauthorized disruptive action in benign episodes and 1.6% under injected text, far fewer than the smaller models (5–6%). DeepCTI+P3 stayed at 0% in both conditions; under forged trusted groups its

rates (3.1–4.7%) equal those of the other models, as the guarantee’s boundary predicts. GLM-4.5-Air (106B MoE), added as a tenth model, reproduces the pattern. Its ReAct loss is 0.97/1.48 on D1 (tracker/withheld) and 0.84/0.96 on D7, against DeepCTI’s 0/0.07 and 0.02/0.34. On D7 upgrades without restart, its DCv21 loss is 0.025, against 5.06 for DC and 3.53 for ReAct.

VI. Discussion and Limitations

What carries the result. The decisive ingredient is not the LLM but the evidence discipline:

- only trusted, independent, fresh observations change a decision;
- missing or conflicting evidence yields an explained abstention;
- disruptive actions need corroborated support.

This turns model quality from a safety variable into a coverage variable: under LTT, weaker models release fewer decisions, but they do not miss more. The LLM earns its place where evidence is unstructured, through verified extraction, and through risk-controlled release of hint-based decisions. An honest reading of H9 is therefore “evidence-state controller versus ReAct”, not “better reasoning”.

Costs.

- Abstention costs coverage: 0.31 on D7 without a feed, before LTT.
- Conservative trust discards useful sources when estimated on little data (repaired in H17).
- Verification trades some benign-data coverage for precision.
- Corroboration counts groups, not physical artefacts. With Maven scanners trusted (H17), a scanner and a direct jar read are distinct groups, although both read the same jar metadata.
- Instance-awareness increases abstention after restarts and adds 34–140% prompt tokens.
- Transfer: on VEX-Bench (62 of 75 tasks), our evidence-state wrapper did not beat the benchmark’s plain harness (status F1 0.613 vs. 0.648 with Gemma-31B; all paired CIs contain 0) and abstained on 54–73% of tasks.

Each of these costs is measured above, and none was tuned on test data.

Threats to validity.

- Tools are simulated over the fixtures. There were no containers, and no host software was executed; only the scanners ran on the file systems.
- Labels come from trackers and advisories without a human audit; tracker lag and errors propagate into them.
- Construction variants are label-correlated, but they are never shown to systems.
- The attack study is weak (§V-E), and an adaptive-attack evaluation remains open.

- Note quality is judged by LLMs, validated only by error injection.
- Nemotron-49B’s ReAct runs used a generic tool template, and 25.5% of its D1 ReAct episodes made no tool call.
- We did not run a practitioner study.

VII. Conclusion

Mitigation decisions should be made by an explicit evidence state, not inside an LLM’s context window. DeepCTI implements this with a provenance-tracked Belnap state, a VEX decision table, value-of-information acquisition, verified extraction, risk-controlled release and evidence-conditioned authorization. Across two live-fixture benchmarks, ten models and four pre-registered rounds, it avoids the missed vulnerabilities that dominate LLM-agent loss in the main studies (drift remains harder). It confines the model to roles where it can be checked, and it blocks unauthorized disruptive actions. The failures we found were a restart-blind controller, conservative trust and a defective synthesis prompt. Each was diagnosed in the data, and each repair was then tested on held-out data under a new pre-registration.

References

- [1] N. Dissanayake, A. Jayatilaka, M. Zahedi, and M. A. Babar, “An empirical study of automation in software security patch management,” in Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, 2022. [Online]. Available: <https://doi.org/10.1145/3551349.3556969>
- [2] A. Jenkins, L. Liu, M. Wolters, and K. Vaniea, “Not as easy as just update: Survey of system administrators and patching behaviours,” in Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems, 2024. [Online]. Available: <https://doi.org/10.1145/3613904.3642456>
- [3] M. Souppaya and K. Scarfone, “Guide to enterprise patch management planning: Preventive maintenance for technology,” National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-40 Revision 4, 2022. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-40r4>
- [4] OASIS, “Common security advisory framework version 2.0,” OASIS Standard, 2022.
- [5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in International Conference on Learning Representations, 2023. [Online]. Available: <https://arxiv.org/abs/2210.03629>
- [6] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in Advances in Neural Information Processing Systems, vol. 36, 2023. [Online]. Available: <https://arxiv.org/abs/2302.04761>
- [7] S. Mitra, S. Neupane, T. Chakraborty, S. Mittal, A. Piplai, M. Gaur, and S. Rahimi, “LocalIntel: Generating organizational threat intelligence from global and local cyber knowledge,” arXiv preprint arXiv:2401.10036, 2024. [Online]. Available: <https://arxiv.org/abs/2401.10036>
- [8] M. T. Alam, D. Bhusal, L. Nguyen, and N. Rastogi, “CTIBench: A benchmark for evaluating LLMs in cyber threat intelligence,” in Advances in Neural Information Processing Systems, Datasets and Benchmarks Track, 2024. [Online]. Available: <https://arxiv.org/abs/2406.07599>
- [9] M. T. Alam, D. Bhusal, S. Ahmad, N. Rastogi, and P. Worth, “AthenaBench: A dynamic benchmark for evaluating LLMs in cyber threat intelligence,” arXiv preprint arXiv:2511.01144, 2025, version 2, February 2026. [Online]. Available: <https://arxiv.org/abs/2511.01144v2>
- [10] Y. Cheng, Y. Liu, C. Li, D. Song, and P. Gao, “CTIConnect: A benchmark for retrieval-augmented LLMs over heterogeneous cyber threat intelligence,” arXiv preprint arXiv:2510.11974, 2025, version 2, June 2026; accepted to KDD 2026. [Online]. Available: <https://arxiv.org/abs/2510.11974v2>
- [11] Y. Wu, M. Velazco, A. Zhao, M. R. M. Luján, S. Movva, J. K. Roy, Q. Nguyen, R. Rodriguez, Q. Wu, M. Albada, Y. Kiseleva, and A. Mudgerikar, “ExCyTIn-Bench: Evaluating LLM agents on cyber threat investigation,” arXiv preprint arXiv:2507.14201, 2025, version 3, May 2026; accepted to ICML 2026. [Online]. Available: <https://arxiv.org/abs/2507.14201v3>
- [12] V. Malamas, V. Paklatzis, C. Douligeris, and D. Koutras, “Can we trust cyber defence agents? TrustOps for evidence grounded security operations,” SSRN preprint, <https://doi.org/10.2139/ssrn.7346308>, 2026, posted August 26, 2026. [Online]. Available: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=7346308
- [13] N. Dissanayake, M. Zahedi, A. Jayatilaka, and M. A. Babar, “Why, how and where of delays in software security patch management: An empirical investigation in the healthcare sector,” Proceedings of the ACM on Human-Computer Interaction, vol. 6, no. CSCW2, pp. 1–29, 2022. [Online]. Available: <https://doi.org/10.1145/3555087>
- [14] F. Li and V. Paxson, “A large-scale empirical study of security patches,” in Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, 2017, pp. 2201–2215. [Online]. Available: <https://doi.org/10.1145/3133956.3134072>
- [15] L. Johnson, K. Dempsey, R. Ross, S. Gupta, and D. Bailey, “Guide for security-focused configuration management of information systems,” National Institute of Standards and Technology, Tech. Rep. NIST SP 800-128, 2011, includes updates as of October 10, 2019. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/128/upd1/final>
- [16] K. Dempsey, N. Chawla, L. Johnson, R. Johnston, A. Jones, A. Orebaugh, M. Scholl, and K. Stine, “Information security continuous monitoring (ISCM) for federal information systems and organizations,” National Institute of Standards and Technology, Tech. Rep. NIST SP 800-137, 2011. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/137/final>
- [17] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [18] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG: Learning to retrieve, generate, and critique through self-reflection,” in International Conference on Learning Representations, 2024. [Online]. Available: <https://openreview.net/forum?id=hSyW5go0v8>
- [19] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, “Corrective retrieval augmented generation,” arXiv preprint arXiv:2401.15884, 2024. [Online]. Available: <https://arxiv.org/abs/2401.15884v3>
- [20] B. He, N. Chen, X. He, L. Yan, Z. Wei, J. Luo, and Z.-H. Ling, “Retrieving, rethinking and revising: The chain-of-verification can improve retrieval augmented generation,” in Findings of the Association for Computational Linguistics: EMNLP 2024, 2024, pp. 10371–10393. [Online]. Available: <https://aclanthology.org/2024.findings-emnlp.607/>
- [21] T. Vu, M. Iyyer, X. Wang, N. Constant, J. Wei, J. Wei, C. Tar, Y.-H. Sung, D. Zhou, Q. Le, and T. Luong, “FreshLLMs: Refreshing large language models with search engine augmentation,” in Findings of the Association for Computational Linguistics: ACL 2024, 2024, pp. 13697–13720. [Online]. Available: <https://aclanthology.org/2024.findings-acl.813/>

- [22] A. Özer and Ç. Yıldız, “Question answering under temporal conflict: Evaluating and organizing evolving knowledge with LLMs,” arXiv preprint arXiv:2506.07270, 2025. [Online]. Available: <https://arxiv.org/abs/2506.07270>
- [23] X. Nie, H. Li, S. Wu, W. Fan, Q. Song, and W. Jiang, “EvoTrustRAG: Evolution-aware conflict attribution and evidence handling for reliable retrieval-augmented generation,” arXiv preprint arXiv:2608.07933, 2026. [Online]. Available: <https://arxiv.org/abs/2608.07933>
- [24] G. Hong, J. Kim, J. Kang, S.-H. Myaeng, and J. J. Whang, “Why so gullible? enhancing the robustness of retrieval-augmented models against counterfactual noise,” in Findings of the Association for Computational Linguistics: NAACL 2024, 2024, pp. 2474–2495. [Online]. Available: <https://aclanthology.org/2024.findings-naacl.159/>
- [25] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, “MemGPT: Towards LLMs as operating systems,” arXiv preprint arXiv:2310.08560, 2023. [Online]. Available: <https://arxiv.org/abs/2310.08560>
- [26] J. H. Saltzer and M. D. Schroeder, “The protection of information in computer systems,” Proceedings of the IEEE, vol. 63, no. 9, pp. 1278–1308, 1975. [Online]. Available: <https://web.mit.edu/saltzer/www/publications/protection/index.html>
- [27] V. Hu, D. Ferraiolo, R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, and K. Scarfone, “Guide to attribute based access control (ABAC) definition and considerations,” National Institute of Standards and Technology, Tech. Rep. NIST SP 800-162, 2014, includes updates as of August 2, 2019. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/162/upd2/final>
- [28] V. Hu, D. Ferraiolo, and R. Kuhn, “Attribute considerations for access control systems,” National Institute of Standards and Technology, Tech. Rep. NIST SP 800-205, 2019. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/205/final>
- [29] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, “Progent: Securing AI agents with privilege control,” arXiv preprint arXiv:2504.11703, 2025, version 3, May 2026. [Online]. Available: <https://arxiv.org/abs/2504.11703v3>
- [30] H. Wang, C. M. Poskitt, and J. Sun, “AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents,” in Proceedings of the 48th IEEE/ACM International Conference on Software Engineering, 2026, pp. 2938–2950. [Online]. Available: <https://arxiv.org/abs/2503.18666>
- [31] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, “Defeating prompt injections by design,” arXiv preprint arXiv:2503.18813, 2025. [Online]. Available: <https://arxiv.org/abs/2503.18813>
- [32] H. Li, X. Liu, H.-C. Chiu, D. Li, N. Zhang, and C. Xiao, “DRIFT: Dynamic rule-based defense with injection isolation for securing LLM agents,” arXiv preprint arXiv:2506.12104, 2025, version 3, March 2026. [Online]. Available: <https://arxiv.org/abs/2506.12104v3>
- [33] V. Siu, J. He, K. Montgomery, Z. Wang, N. Gong, C. Wang, and D. Song, “A framework for formalizing LLM agent security,” arXiv preprint arXiv:2603.19469, 2026. [Online]. Available: <https://arxiv.org/abs/2603.19469>
- [34] Y. Louck, “Securing LLM-agent long-term memory against poisoning: Non-malleable, origin-bound authority with machine-checked guarantees,” arXiv preprint arXiv:2606.24322, 2026.
- [35] A. Pandey, A. Jain, L. Chen, C. Maple, and C. Panchev, “Certified multi-source integrity for structured agent actions,” arXiv preprint arXiv:2609.34245, 2026.
- [36] J. W. Cutler et al., “Cedar: A new language for expressive, fast, safe, and analyzable authorization,” Proceedings of the ACM on Programming Languages, vol. 8, no. OOPSLA1, 2024.
- [37] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” in Advances in Neural Information Processing Systems, 2024. [Online]. Available: <https://arxiv.org/abs/2406.13352>
- [38] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents,” in Findings of the Association for Computational Linguistics: ACL 2024, 2024, pp. 10 471–10 506. [Online]. Available: <https://aclanthology.org/2024.findings-acl.624/>
- [39] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, “Identifying the risks of LM agents with an LM-emulated sandbox,” in International Conference on Learning Representations, 2024. [Online]. Available: <https://arxiv.org/abs/2309.15817>
- [40] T. Yuan, Z. He, L. Dong, Y. Wang, R. Zhao, T. Xia, L. Xu, B. Zhou, F. Li, Z. Zhang, R. Wang, and G. Liu, “R-Judge: Benchmarking safety risk awareness for LLM agents,” in Findings of the Association for Computational Linguistics: EMNLP 2024, 2024, pp. 1467–1490. [Online]. Available: <https://aclanthology.org/2024.findings-emnlp.79/>
- [41] R. Bhagwatkar, K. Kasa, A. Puri, G. Huang, I. Rish, G. W. Taylor, K. D. Dvijotham, and A. Lacoste, “Indirect prompt injections: Are firewalls all you need, or stronger benchmarks?” arXiv preprint arXiv:2510.05244, 2025, version 2, March 2026. [Online]. Available: <https://arxiv.org/abs/2510.05244v2>
- [42] D. Golovin, A. Krause, and D. Ray, “Near-optimal Bayesian active learning with noisy observations,” in Advances in Neural Information Processing Systems (NeurIPS), 2010.
- [43] D. Golovin and A. Krause, “Adaptive submodularity: Theory and applications in active learning and stochastic optimization,” Journal of Artificial Intelligence Research, vol. 42, pp. 427–486, 2011.
- [44] A. N. Angelopoulos, S. Bates, E. J. Candès, M. I. Jordan, and L. Lei, “Learn then test: Calibrating predictive algorithms to achieve risk control,” arXiv preprint arXiv:2110.01052, 2021.
- [45] Y. Wu and V. Rus, “Safe to stop? Risk-constrained stopping for sequential clinical diagnosis agents,” arXiv preprint arXiv:2609.09678, 2026.
- [46] “VEX-Bench,” <https://github.com/steven1518/vex-bench>, commit 80cddea; arXiv:2609.08040, 2026.
- [47] N. D. Belnap, “A useful four-valued logic,” in Modern Uses of Multiple-Valued Logic, J. M. Dunn and G. Epstein, Eds. Reidel, 1977, pp. 5–37.
- [48] UK AI Security Institute, “Inspect AI: Framework for large language model evaluations,” https://github.com/UKGovernmentBEIS/inspect_ai, 2024.
- [49] S. Holm, “A simple sequentially rejective multiple test procedure,” Scandinavian Journal of Statistics, vol. 6, no. 2, pp. 65–70, 1979.